

Internship Report at H Company

Improving episodic memory in computer-use agents

Alessandro Pranzo

September 7, 2026

Contents

1	Introduction	3
1.1	Memory types in AI agents	3
1.2	Why episodic memory: a motivating example	4
1.3	Contributions	4
2	Related work	6
3	Episodic memory system	6
4	Architectures	7
4.1	Flywheel architecture	7
4.2	Policy architecture	8
5	Training recipe	10
5.1	Flywheel collection	10
5.2	Trace filtering and post-processing	12
5.3	SFT post-training	14
6	Evaluation protocol	15
6.1	Dataset selection	15
6.2	Five-phase setup	17
7	Results	17
7.1	Learning across the five phases	18
7.2	Generalisation to unseen tasks and applications	20
7.3	Learning to use the memory tools	21
7.4	Tip quality	24
8	Ablations	26
8.1	Flywheel architecture design	26
8.2	Initial memory briefing	27
8.3	Round 1 versus round 2	28
9	Limitations	30
10	Conclusion and future work	30

A	Full rollouts of the motivating example	33
B	Training details and loss curves	35
C	Flywheel iterations	37
D	Conversion example	38
E	Tip texts	39

1 Introduction

Computer-use agents are agents built on visual language models (VLMs) that complete tasks by interacting with a Graphical User Interface (GUI) environment. Ideally, they extend the abilities of standard AI agents by navigating, scrolling and interacting with visual digital elements just like a human using a computer would. These agents operate by iteratively reasoning about tasks, selecting actions, executing them, and observing results. However, like other AI agents, computer-use agents learn only during training, thus, once deployed to users, they are unable to learn from real-world scenarios that were not seen during training. This limitation of AI agents at “learning on the job” is usually referred to as *continual learning*. The next frontier of AI agents will be dominated by agents that are able to improve their performance as a function of the tasks they have solved, hence learning from experience. We are starting to notice early approaches to continual learning in mainstream agents harnesses like Hermes Agent [13], Claude Code [2] and OpenClaw [11]. In all these cases learning from experience is considered as a memory problem, *i.e.* “how can an agent remember, from its own execution history, what worked and what did not?” For example, an agent that successfully recovers from an error in a run should remember the solution once it finds a similar error in another run.

Here we present a framework to teach agents how to learn continually by using memory tools that enhance their continual learning abilities. Our contributions include:

- Designing an evaluation framework that quantifies the ability of agents to learn from past experiences at test-time.
- Introducing a flywheel agent harness to collect useful data for fine-tuning and memory tools for the agent policy to use.
- Devising a filtering technique to select good examples of continual learning.
- Constructing a training recipe to automatically improve continual learning abilities of the agents.

1.1 Memory types in AI agents

According to [20] there are four different types of memory in AI agents.

Working Memory. The agent’s short-term store, holding the immediate context: the active prompt, user input, and whatever workspace state is currently relevant. It’s what lets the agent reason and decide in the moment, within the same run.

Semantic Memory. Is the memory that is associated to the external world knowledge. Namely, the date, time and location of the action of the agent as well as information coming from news feeds.

Parametric Memory. Everything that lives in the model’s weights. Updates to those weights encode knowledge that persists across tasks and forms the agent’s general competence, with no exact recollection of past experience.

Episodic Memory. A durable record of past experiences and events. Pos-

sibly containing strategies and recoveries from previous runs.

Continual learning can be pursued through any of these components, most obviously parametric memory, by continuing to train on new experience. This report makes a scoping choice: we study continual learning through *episodic* memory alone, leaving the weights frozen once the agent is deployed and giving it instead a store of its own past runs to read from and write to. What the agent accumulates is therefore experience that it can reference to at runtime.

1.2 Why episodic memory: a motivating example

We provide evidence for the need to solve continual learning in computer-use agents by presenting a simple example in a Linux-based environment. Figure 1 shows five independent rollouts of the same policy on a single spreadsheet-cleaning task in OSWorld [18], of which one succeeds. The five come from the same weights and the same prompt, so the procedure that solves the task was available to all of them. A solution was found once and then thrown away. Because no state is carried from one run to the next, the fifth attempt begins as uninformed as the first, and the cost of rediscovering the shortcut, or of failing to, is paid again on every attempt. The policy can solve this task; what it cannot do is remember that it solved it. Retaining that memory means retaining a specific past episode of the agent’s own behaviour, which is precisely the episodic memory defined in Section 1.1. The remainder of this report is concerned with giving computer-use agents access to their own past runs, and with measuring whether that access changes their behaviour.

1.3 Contributions

Our approach to solving episodic memory is based on implementing a system that allows agents to store, read and delete pieces of information from a memory store. To allow agents to learn how to interact with this memory system we devise the following recipe: implementing a collection harness to collect a set of trajectories of agents that are able to interact with the episodic memory system, designing a filtering technique to select a good sample of trajectories, a conversion tool to translate trajectories coming from the tailor-made harness to trajectories for fine-tuning a policy agent, a supervised fine-tuning (SFT) post-training step to teach the agents to use memory tools.

An additional contribution of our approach includes the setup of an evaluation framework to assess the abilities of agents to effectively and efficiently interact with episodic memory systems.

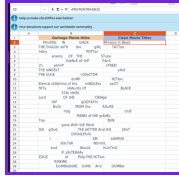
Throughout our analysis we will focus on the *Holo* family of models. *Holo-3.1* is a family of Qwen3.5-based [12] visual language models for computer-use agents, spanning 0.8B, 4B, 9B, 35B-A3B, 122B-A17B parameter scales, trained to operate across web, desktop and mobile GUIs with native function-calling support [3].

✓ RUN 4 • 15 steps



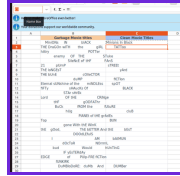
step 4

write =PROPER(TRIM(B2))



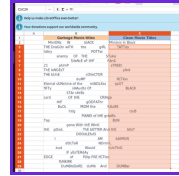
step 7

ctrl+c the formula



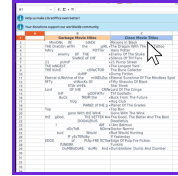
step 10

Name Box: C3:C29



step 11

ctrl+v over the range

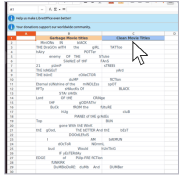


step 14

verify, then save

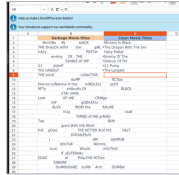
The only successful rollout. One formula, one copy, one range selection through the Name Box, one paste: 28 rows cleaned in 15 steps without transcribing a single title, so no title can be corrupted.

✗ RUN 1 • 59 steps



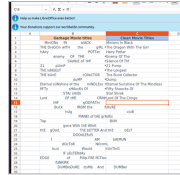
step 1

click cell C2



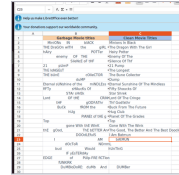
step 17

type row 9 by hand



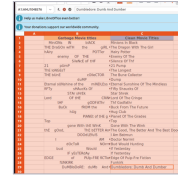
step 29

type row 15 by hand



step 45

type row 24 by hand

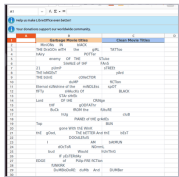


step 59

"task completed"

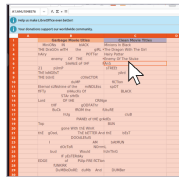
One cell at a time, 28 times. The trailing word is lost in nine rows (Enemy Of The for Enemy Of The Stuke) and If Yesterday is written twice, shifting the tail of the column. The agent still reports success.

✗ RUN 2 • 99 steps



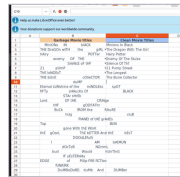
step 1

plan the row loop



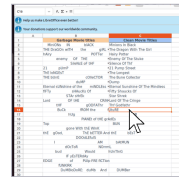
step 22

click C6



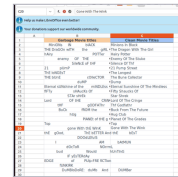
step 50

type row 10 by hand



step 78

double-click C16



step 99

"20/28 rows"

The same per-row loop, interleaved with navigation churn (repeated ctrl+home to relocate). The step budget is spent on row 20 of 28 and the agent concedes a partial result.

Figure 1: The same task, three times, and only one usable strategy. Three of five independent rollouts of the same policy on the OSWorld task libreoffice_calc_40, which asks the agent to write a cleaned copy of the garbage-cased movie titles of column B into column C. Each row is one rollout at five of its steps, showing the spreadsheet as the agent saw it and the action it took next. The top rollout (violet) is the only one of the five that succeeds: it writes a single PROPER(TRIM(...)) formula, copies it, selects C3:C29 through the Name Box and pastes once, finishing in 15 steps. The two below (grey) treat the task as 28 independent edits and walk down the column cell by cell: the first reaches the end but corrupts nine titles on the way and still reports success, the second exhausts its step budget at row 20. All five rollouts are reproduced at a larger size in Appendix A.

2 Related work

Learning from past runs has so far been treated mostly as a context-engineering problem, leaving the policy frozen. The closest work to ours is [4], which mines completed trajectories offline and extracts typed, causally attributed tips, namely strategies, recoveries and optimizations, then deduplicates them and retrieves them per task on AppWorld [15]; the same extract-consolidate-retrieve loop backs production memory stacks [10]. [7] and [5] go further and let the agent rewrite its own harness, memory included, from its trajectory data. In all of these cases the model is never updated, so the decision of when to write a tip and when to read one is delegated to hand-written callbacks, and the failure mode they report is not missing memory but mismatched memory: irrelevant tips actively degrade performance. [19] does train memory operations with reinforcement learning, but it trains a memory manager that is separate from the acting policy, and it does so over conversational facts, where an action costs nothing and a recovery is not worth remembering.

For computer-use agents specifically, [17] compresses each GUI trajectory into a fixed-length sequence of continuous embeddings fed to the input layer of the backbone, which scales with retrieval depth better than text does. That memory, however, is not human-readable, cannot be audited or edited, and requires changing the architecture that serves the model. On the evaluation side, [16] measures the improvement over a stream of tasks rather than the accuracy on any single one, yet its observations are text and API responses, so no existing benchmark tells us whether a computer-use agent gets better at GUI tasks as it accumulates experience.

Our work addresses these three limitations: we train the policy itself to fire the memory tools instead of relying on callbacks, we keep the tips in natural language so that they stay inspectable and attributable to the run that produced them, and we measure continual improvement in a GUI setting through our five-phase evaluation.

3 Episodic memory system

The episodic memory system that we have implemented for our approach is composed of few simple fundamental elements. The atomic element of this system is a **Tip**. Tips are brief and short piece of guidance that agents decide to extract while performing a task. An example of a Tip is:

Clean text with PROPER and TRIM in LibreOffice Calc

To clean messy text in LibreOffice Calc by removing redundant white spaces and converting to title case (first letter of each word capitalized, rest lowercase): Use the formula `=PROPER(TRIM(cell_reference))`. `TRIM()` removes leading, trailing, and extra internal spaces. `PROPER` capitalizes the first letter of each word and lowercases the rest. Enter the formula in the first cell, then copy and paste down to fill all rows. The formula automatically adjusts references for each row.

The main idea behind Tips is to construct a way for the agents to distill relevant information to solve a specific task in a short piece of text that can later be

retrieved and included in the context of another agent that performs a similar task to increase its likelihood of success.

Tips created by an agent are collected in a memory store (we call it **tip store**) backed by a NextPlaid server [14], a multi-vector search engine that indexes each tip with a ColBERT late-interaction model [8] and serves semantic retrieval over a REST API. Late interaction is chosen over a dense single-vector encoder for retrieval quality: scoring a query against every token embedding of a document, rather than against one pooled vector, consistently ranks higher than dense bi-encoders at comparable index cost [8]. Documents can be added and deleted incrementally, so extraction and deletion never require rebuilding the index. This store selection naturally requires us to define three main operations that agents can use to interact with it:

- **Extraction:** the operation of *extracting* a relevant Tip from an agent run and adding it to the memory store.
- **Injection:** the operation that retrieves relevant tips from the memory store; based on the current state of the agent. The retrieved Tips get *injected* in the context of the agent.
- **Deletion:** the operation that allows the agents to modify the memory store by removing the Tips that the agents believe to be harmful in later runs.

Once given these operations, the agents are able to manage their own episodic memory, potentially allowing them to maintain an experience memory that incrementally improves their performance over tasks.

4 Architectures

In order to teach the agents to learn how to interact with the episodic memory store, we need to collect a dataset of trajectories where these interactions are performed. We design a harness that allows us to collect such trajectories in a scalable way without the need of training a policy model to use the interaction tools. Therefore, in this section we will describe the architecture that allows us to collect such trajectories with no training required and also the architecture of the policy model that will be used for training later further down the pipeline.

4.1 Flywheel architecture

Figure 3 shows that the flywheel architecture is a two-model architecture that allows us to collect a dataset of trajectories where the agent interacts with the episodic memory store. The main model is the agent itself which is responsible for interacting with the environment just like a standard policy would do. As a callback at each step there is a second and detached large language model (LLM) agent: the manager. The manager is responsible for taking the state of the main agent at that step and the previous decisions of the manager itself and returning a single decision: inject a tip into the context, create one, delete one, or pass (do nothing). The only memory read that is not decided by the

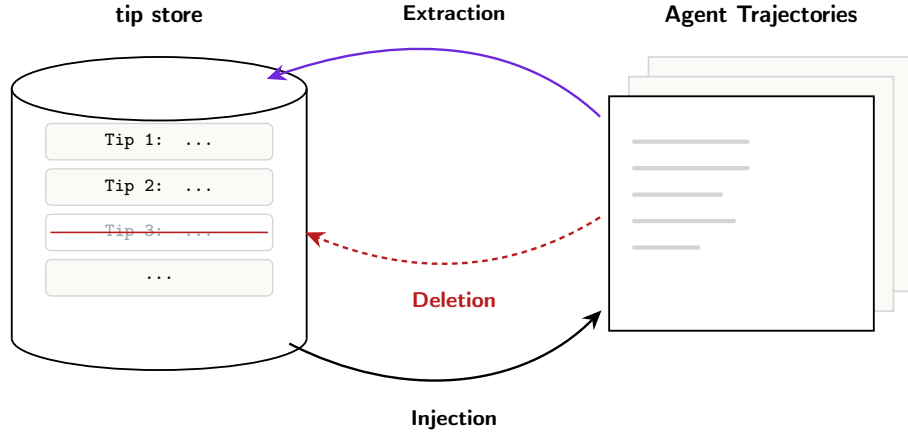


Figure 2: **The three operations of episodic memory.** Completed trajectories are mined for reusable advice, which is written to a store of natural-language tips (**extraction**). Tips are read back into the context of later runs (**injection**), and a tip that a later run followed to a worse outcome is struck from the store (**deletion**), so the store is corrected by use rather than only grown by it. The store is the only state that survives between runs, and it stays human-readable, so every tip can be traced back to the trajectory that produced it.

manager is the briefing, forced by the harness at step 0. The manager acts as a classifier that generates a signal of a potential memory operation that the agent should perform for each step. The upsides of this architecture are that it allows us to collect a dataset of trajectories where the agent interacts with the episodic memory store without the need of training a policy model to use the interaction tools. On the downside this architecture is heavy in terms of computational resources and time as it requires at least two model calls per step.

4.2 Policy architecture

In Figure 4 we present the policy architecture which is a single-model architecture that allows the acting agent itself to interact with the episodic memory store. Unlike the flywheel, there is no second detached manager: the same policy that operates the GUI is also the one that reads and writes tips, and the three memory operations become ordinary tool calls sitting in the same action list as the browser actions. At each step the agent can search the store (**search_tips**), save a new tip (**save_tip**), delete an existing one (**delete_tip**), or simply act in the environment. Firing a memory tool consumes a step that could otherwise have advanced the task, so the policy has to learn not only how to use the tools but when it is worth doing so. The only memory read the policy does not choose is the briefing, still forced by the harness at step 0. the policy is a tool-user rather than a classifier, which is the behaviour we later teach with the training recipe.

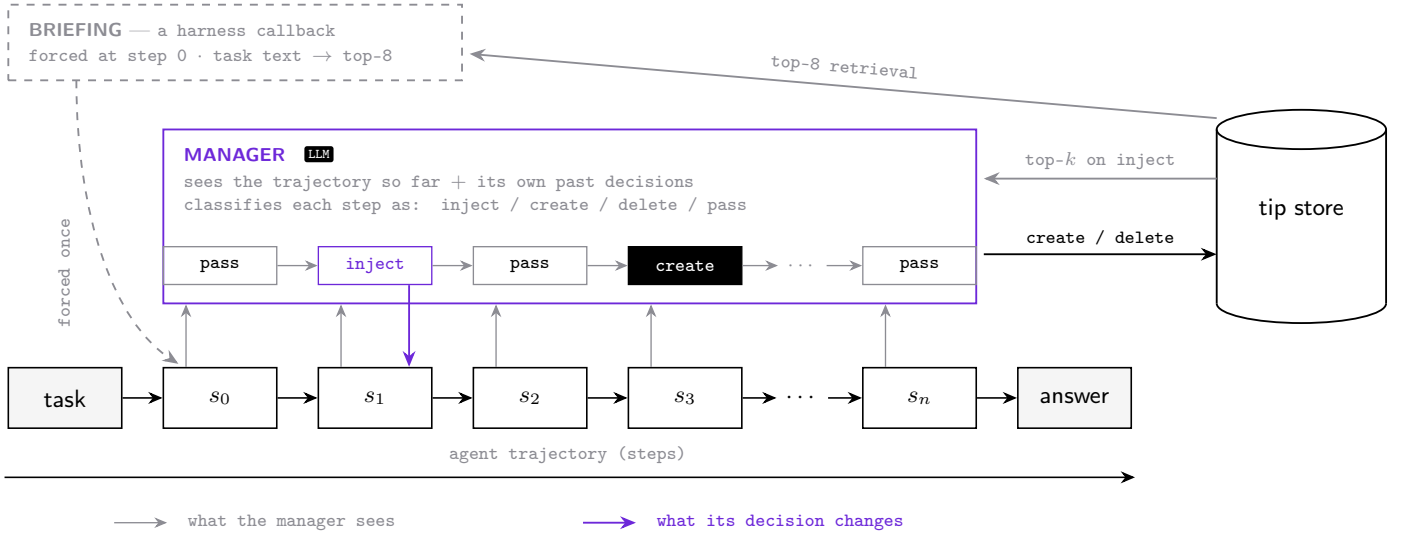


Figure 3: **The flywheel architecture.** The underlying policy from the task produces a sequence of steps and an answer. Two mechanisms read and write a shared store of natural-language tips around it. The *briefing* fires once, before the first step, retrieving the top- k tips with the task text itself as the query, so every rollout starts from whatever the store already knows. The *manager* is a second model, invoked once per step. It is given the trajectory up to that step together with its own earlier decisions and returns a single label: inject a tip into the context, create one, delete one, or pass. It is a classifier rather than a tool-user: it never acts in the environment and emits exactly one decision per step, which is what makes its behaviour cheap to supervise.

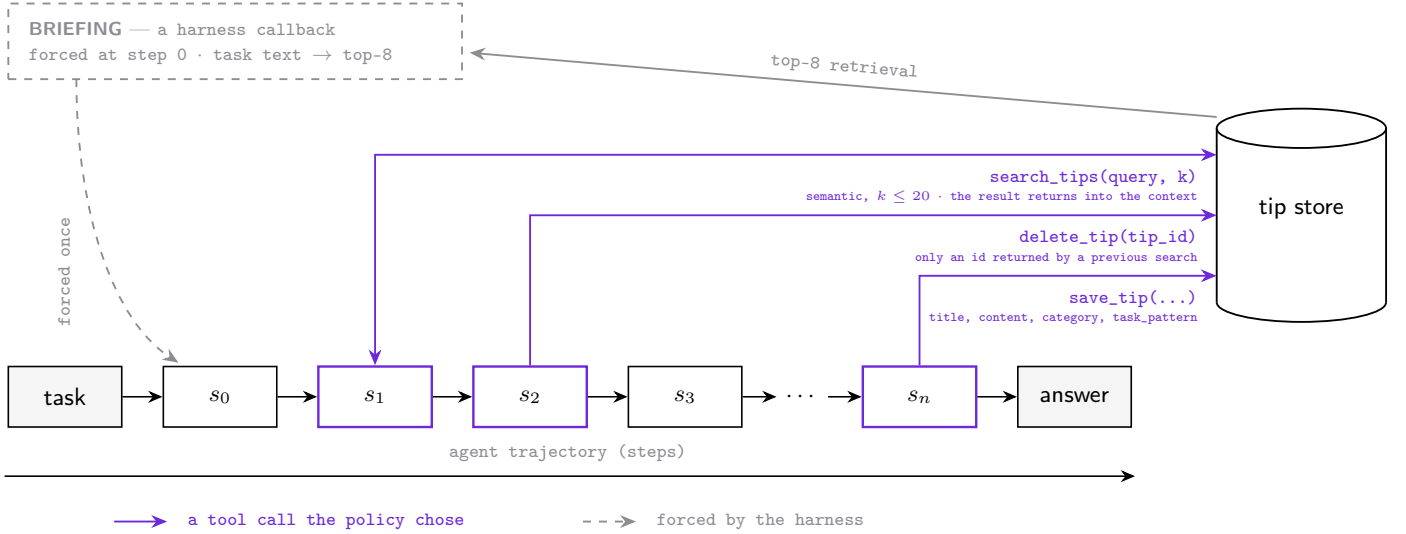


Figure 4: **The policy architecture.** The same rollout and the same store, with the memory operations moved into the policy’s own action space. The three violet operations are ordinary tool calls sitting in the same action list as the computer-use actions: the policy decides whether to issue one and at which step, and each call consumes a step that could instead have advanced the task. The briefing itself is the one memory read the policy does not choose.

5 Training recipe

The architectures of Section 4 describe two ways of attaching a tip store to a rollout, but only one of them is a deployable agent. The flywheel harness needs a second model at every step, which doubles the inference cost of a run and makes memory a property of the harness rather than of the policy. The policy architecture removes that model, at the price of requiring the policy to have learned when a memory operation is worth a step, a behaviour an un-fine-tuned model does not exhibit. The recipe below closes that gap: it uses the flywheel harness as a *teacher* to generate memory decisions, keeps only the decisions that turned out to be worth imitating, and relabels them as the tool calls the policy will have to make itself. Figure 5 shows the whole pipeline.

5.1 Flywheel collection

Collection runs the flywheel harness of Figure 3 over the training split, with a 35B policy and a 122B manager. The manager is deliberately the larger of the two: it is the source of the supervision signal and is discarded after collection, so its cost is paid once rather than at every deployment.

Task pool and split. The pool is the union of the two learnable subsets of Figure 6 giving 890 tasks, 84 from OSWorld and 806 from WebArena Infinity. It is split once, stratified by benchmark, application and difficulty, into 712 training and 178 held-out tasks. Collection never touches the held-out side, so no rollout of the flywheel is collected on this side of the split.

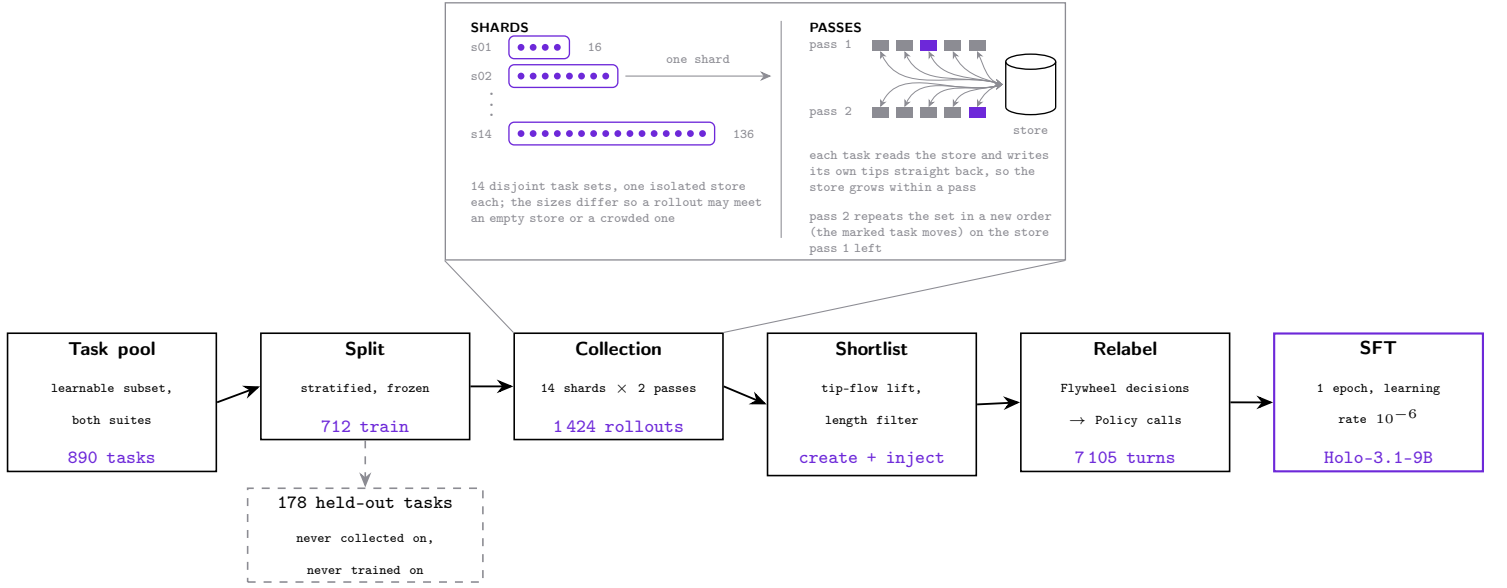


Figure 5: **The training recipe.** The pool is the union of the two learnable subsets of Figure 6; the split is drawn once and frozen, and the held-out side is never collected on. Collection runs the flywheel harness of Figure 3, whose manager decisions are afterwards relabelled as the tool calls of Figure 4. The inset expands the collection stage. On the left, each shard is the set of training tasks it holds; the 14 sets are disjoint, hold a store apiece. On the right, one of those shards is opened up: its task set is replayed twice in a fresh random order, and the two-headed arrows record that every task both reads the shard’s store and writes its own tips straight back into it.

Shards. We would like to simulate a heterogeneous environment with different tasks having different stores states. For some tasks we would like injections to draw tips from almost empty states of the tip stores, while for others we would like them to be more crowded; ideally spanning a wide range of different states of the tip store that resembles a real world application. Thus, the training tasks are partitioned into 14 shards, each with its own tip store. Shards are independent, so they run concurrently without interacting. Shard sizes deliberately vary, from 16 to 136 tasks, to capture the full spectrum of environment states: from a sparse regime, where creating tips is the primary action, to a crowded regime, where selecting and pruning tips becomes critical.

Passes and store dynamics. Each shard is traversed twice, in a fresh random order each time, so a given task is sampled at two decorrelated store states rather than always early or always late. Unlike the evaluation protocol of Figure 7, writes here are applied *inline*: a tip created by one task is immediately visible to the tasks running beside it. The store a trajectory saw is therefore stochastic. In total the campaign produces $712 \times 2 = 1\,424$ rollouts.

5.2 Trace filtering and post-processing

A rollout produced by the flywheel is not automatically worth imitating: the manager fires on every step, and most of what it does is either a pass or a decision that led nowhere. We need to filter out the rollouts that are not worth imitating, to do so we would need to assign a score to each memory decision based on their influence on other rollouts that are affected by such decisions.

We devise a score for each tip generated in a campaign. Every tip that is created and later delivered to another attempt constructs an edge in a graph over the collection campaign, we call this analysis a tip flow. A delivery of a tip to a receiving attempt v is weighted by how much better that attempt did compared to the no-memory baseline for its task, with credit shared among the tips the receiver saw:

$$w(v) = \frac{S(v) - b(\tau(v))}{k(v)}, \quad q(t) = \sum_{v \in \text{deliveries}(t)} w(v), \quad (1)$$

where $S(v) \in \{0, 1\}$ is the receiver’s outcome, $b(\tau(v))$ the baseline pass rate of its task over the coverage sweep in Figure 6, and $k(v)$ the number of tips that reached it (the more injections a task receives the least credit an individual tip gets). Deletions are excluded, since a deletion is not an exposure. A tip’s quality $q(t)$ is the sum of its delivery weights, so a tip is good to the extent that the attempts which received it beat what those tasks were expected to score without memory.

Based on the scores that we assigned through Equation 1, we shortlist two behaviours from the 1 424 rollouts: *Extraction* is judged only on the downstream value of the tip produced, regardless of whether the rollout performing it succeeded or not, a run can fail and still leave the right lesson behind (maybe tips to avoid certain actions). We therefore score extractors on q_{cross} , with same-

task deliveries removed, and additionally require at least two distinct receivers so that no tip is promoted on a single outcome. *Injection* is judged on both the retrieval and the use: the attempt received a tip and went on to succeed. Here the receiver’s own outcome inflates the score of every tip it saw, so a leave-one-out variant q_{loo} , computed with all deliveries to that receiver removed, supplies the honest reading. In both cases a tip is counted as positive when its score is strictly greater than zero.

Admitting failed rollouts is intentional, however, during training, we expand each example per-turn. This weights each admitted rollout by its own length, and failures are the long ones. In the first collection round 38 of the 321 short-listed rollouts had failed (11.8%), yet they contributed 20.7% of the training examples, averaging 43.1 steps against 22.2 for the successful ones. A model trained on that mixture, as we will see in Section 8.3 searches in loops and saves many different tips. The second round therefore drops any failed rollout longer than twice the median successful length, while keeping the short failures, which remain the honest source of recovery and avoidance tips. This is the only difference between the two dataset rounds of Table 7.

Relabelling. A collected flywheel rollout is a recording of two actors. The agent clicked, typed and acted with the environment; the manager watched it and, at every step, decided whether memory should be touched. What we need to train is a single actor that does both, so the collected trace must be rewritten to attribute the manager’s decisions to the agent itself.

First, a decision becomes an action: where the manager decided to retrieve, the agent is shown calling `search_tips`; where it decided to write a tip, the agent is shown calling `save_tip` with that tip’s text; and likewise for `delete_tip`.

Second, the sentence explaining the decision changes voice. The memory manager wrote about the agent from the outside — “*the agent successfully located the setting*” — and a model that has to perform memory actions itself should rather reason over its own actions in first person. A second, small language model (`qwen3-5-4b`) rewrites each of those sentences into the first person, under instructions to keep every concrete detail, to invent nothing, and to state an intention rather than deliver a verdict. This is the only place in the pipeline where a model writes text that becomes a training target, so two properties are enforced: each rewrite is keyed by a hash of its source and cached, meaning a re-run of the conversion reproduces the previous dataset exactly rather than resampling. A converted rollout is therefore indistinguishable in shape from one the agent produced itself, and nothing in the format signals to the model that these examples were assembled after the fact.

Every step carries a short note summarising the screen, and a synthesised memory turn needs one too, in this case we populate with the notes generated by the manager. In Figure 23 we show all the changes applied to a rollout to convert it from the format generated by the flywheel collector to the one needed to train the policy.

What is trainable. In Table 1 we state what parts of the converted rollouts are the ones that are going to be trainable for the SFT training of the model. Retrieved and briefed tips reach the model only as masked observations: the

	Masked (context)	Trainable (target)
Briefing, injected and retrieved tips	✓	
Observations and tool results	✓	
Environment action, note, thought		✓
<code>search_tips</code> query and k		✓
<code>save_tip</code> tip text		✓
<code>delete_tip</code> target id		✓

Table 1: **The label mask.** Tips the agent *reads* are context; tips and queries the agent *writes* are targets.

model must condition on them and must never learn to generate them, which would be both leakage and a train-serve mismatch, since the store differs at evaluation time. The argument of a `save_tip` call is the one place where the model is the author of tip text, and it is trainable. Each converted turn becomes one training example whose trailing assistant message is the target.

5.3 SFT post-training

The converted examples are packed into a single dataset, then used to fine-tune the policy model. In order to avoid catastrophic forgetting [9] of general model abilities we deploy a very conservative training schedule (low learning rate and few optimization steps). Table 7 gives the two dataset rounds and Table 8 the training configurations.

The objective. Training minimises the ordinary next-token cross-entropy, evaluated only at the positions the mask of Table 1 leaves trainable:

$$\mathcal{L} = -\frac{1}{|M|} \sum_{i \in M} \log p_{\theta}(x_i \mid x_{<i}), \quad (2)$$

where M is the set of trainable token positions of an example. Everything on the masked side enters only through the conditioning context $x_{<i}$: the briefing, the screenshots, the observations and any retrieved tips are read but never predicted. The sum is normalised by the number of trainable tokens in the example rather than across the batch, so a turn whose target is a two-line `search_tips` call weighs as much as one whose target is a paragraph of saved tip text.

Training statistics. Table 9 reports the three runs we kept and Figure 22 their loss curves. All three are single-epoch runs on eight H100s per node, with a micro-batch of one sequence per GPU and no gradient accumulation, so the optimisation batch is exactly the GPU count and one step consumes one packed sequence per GPU. Round 2 fits in half the steps of round 1 because the length filter removed the long failed rollouts, and it runs on two nodes rather than one, which together take the 9B run from 79 to 37 minutes. The per-step loss is computed on a single sequence, so the raw trace in Figure 22 swings between 0.01 and 0.45 according to which example the step happened to draw; only the smoothed line is worth reading. Against that caveat, the two round-2 runs descend cleanly and flatten by roughly step 150, while the round-1 run flattens near 0.10 and then rises over its last 60 steps.

Long sequences. An episodic-memory example is long: it carries the briefing, the trajectory so far and up to two screenshots. Examples exceeding the maximum length are truncated *from the beginning* rather than the end, because the target is the trailing assistant message; truncating the tail instead left several hundred examples with no trainable tokens at all, silently reducing the effective dataset.

6 Evaluation protocol

Before diving into the architectures of the agents and the training recipe needed to train them at better using episodic memory, we need to devise an evaluation framework to assess the continual learning abilities of computer-use agents.

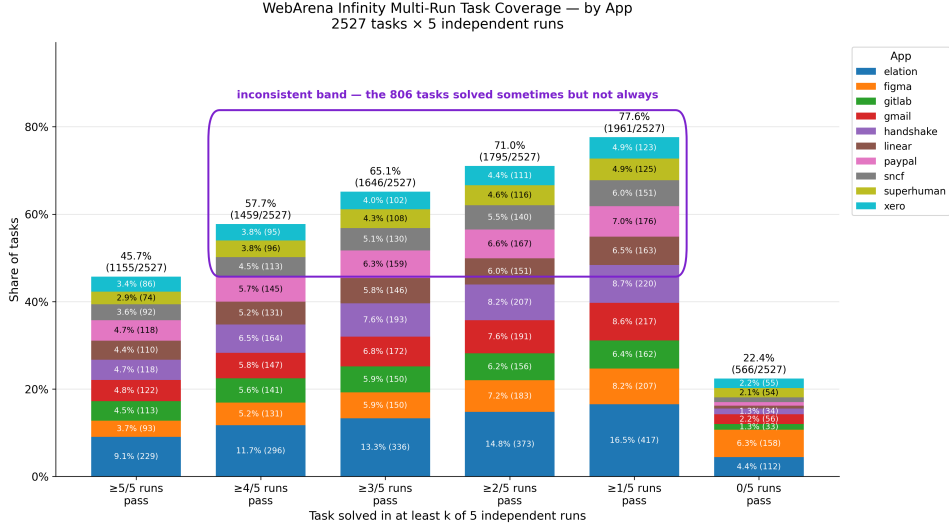
6.1 Dataset selection

The starting point of the evaluation framework is the pool of tasks that we want the model to complete. For computer-use agents we rely on two complementary benchmarks, one for the desktop and one for the browser.

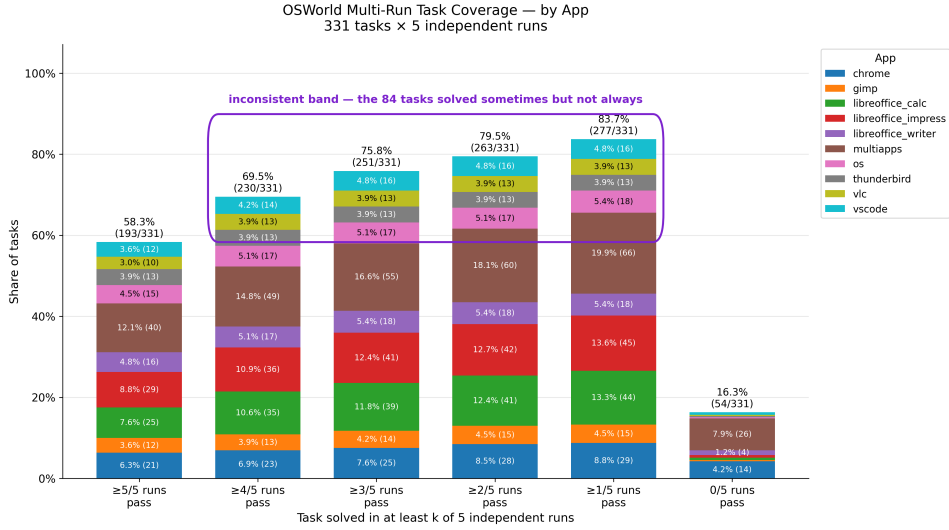
OSWorld [18] is a real computer environment in which multimodal agents complete open-ended desktop tasks using office suites, file I/O, browsers and multi-application workflows by operating the GUI of a live Ubuntu machine, each task scored by a custom execution-based evaluator on the final state of the system. We take its 331-task Ubuntu suite as our desktop pool.

WebArena Infinity [21] is the browser counterpart: a suite of auto-generated web applications cloned from real product documentation, each paired with difficulty-graded tasks whose success is checked programmatically against application state rather than by a human judge. The snapshot we use contains 2 527 verifiable tasks spanning environments such as Gmail, GitLab, PayPal and Linear.

We choose to create a subset out of this shared pool of tasks. Ideally we think that tasks that can be solved by default by the models only a fraction of the times (like the one we mentioned in Figure 1) would help us assess if the models learn from experience. A task the policy always solves leaves nothing for experience to improve; a task it never solves gives the store nothing useful to remember. To select the right subset, we therefore attempt every task five times independently with the same frozen policy and keep only the inconsistent band of Figure 6: the tasks solved in at least one run and failed in at least one other. That filter leaves 84 OSWorld tasks and 806 WebArena Infinity tasks, and the five-phase evaluation of the next section runs on that band. Both halves of it feed collection, but only the browser half is carried through to Section 7: the fine-tuned policy is never evaluated on the desktop band, for the reason given in Section 9. The desktop tasks still appear in Section 8, where the flywheel harness itself is measured on them.



(a) WebArena Infinity: 2527 browser tasks, 806 in the band.



(b) OSWorld — 331 desktop tasks, 84 in the band.

Figure 6: **Execution variance on both benchmarks.** Every task was attempted five times independently by the same policy; each bar gives the share of tasks solved in at least k of the five runs, split by the application the task targets, and the rightmost bar holds the tasks no run solved. Reading a panel left to right separates competence from reliability. On WebArena Infinity (a), 45.7% of tasks are solved in all five attempts against 77.6% solved at least once; on OSWorld (b) the always-solved share is higher (58.3%) and the never-solved share lower, yet the spread between the two levels is comparable, so the suites differ in difficulty rather than in variance. The **bracketed** band is that spread: the tasks solved sometimes and not always, which is the subset the episodic-memory evaluation runs on.

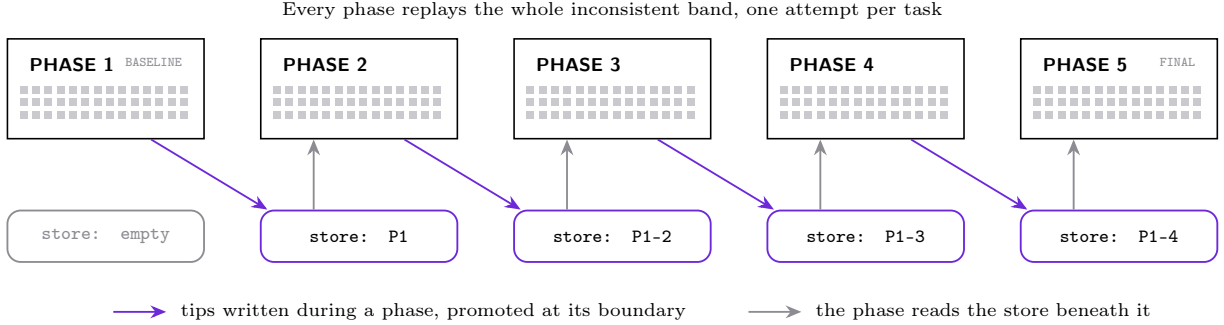


Figure 7: **The five-phase evaluation protocol.** Every phase replays the same task set, the inconsistent band of Figure 6, one attempt per task, so a rising success rate across phases can only come from what the store accumulated in the phases before. The store drawn beneath a phase is the store that phase reads: Phase 1’s is empty, which makes it the no-memory baseline, and each phase’s own tips are promoted into the store of the phase that follows. In the figure, P_n abbreviates Phase n . A store is frozen for the duration of the phase reading it, so every task in a phase is offered the identical set of tips and the order of tasks cannot matter. Phase 5 therefore reads a store built from four phases, and its own writes are discarded because no phase follows it.

6.2 Five-phase setup

Individually evaluating the subset of tasks that we selected, however, would not yet allow to infer any result regarding the ability for the model to continually learn. As we mentioned in the introduction, continual learning requires the agent to improve its ability to solve a task as a function of the times it encounters it. To evaluate this result in our setup, we decide to construct a sequential five-phase process. In each phase, the agent replays the same subset constructed in Section 6.1, injecting the tips that were added to the store by previous phases of the same agent and extracting new tips that get added to the store at the end of the phase. This setup simulates an agent that completes a diverse set of tasks through time and allows to simply evaluate the effect of its experience on the performance. Ideally we expect an agent that is able to learn continually to increase monotonically its performance over the same number of tasks as phases complete. Additionally, by having a unique store for all the tasks, this framework allows the agent to potentially benefit from the tips it extracts from traces that are completely different from each other.

7 Results

In this section, under the five-phase protocol of Figure 7, we evaluate our approach to teach agents to learn continually on the browser suite, WebArena Infinity, on the 806-task inconsistent band of Section 6.1, replayed. We expect an agent that is able to learn continually to increase its performance over the same set of tasks as a function of the times it completes them. Accordingly, we expect that an agent learns how to work more efficiently, thus reducing its number of steps to complete a task as it improves its experience. We will show

that the methodology developed allows models of different sizes (9B and 4B respectively) manage to achieve continual learning capabilities over the evaluation dataset. We also show their capability to transfer this ability to unseen tasks, while demonstrating that they do so by effectively using the memory tools that we have designed. Finally, we show the motivations that lead to our exact setup, more specifically: the filtering choices, the flywheel architecture and the policy harness design.

7.1 Learning across the five phases

Figure 8 presents the main result. We run five independent runs of the same experiment and collect their results for greater significance. The 9B fine-tune improves monotonically across all five phases, $57.8 \rightarrow 64.3\%$, while the checkpoint it was trained from, on the identical task set, under the identical harness, with the identical budgets, drifts from 55.9% to 54.7%. The 4B pair behaves the same way at a lower level, $34.7 \rightarrow 38.4\%$ against a base that falls $33.8 \rightarrow 32.0\%$. In both pairs the two checkpoints start within roughly a point of each other. As the store fills the gap in performance between the base and fine-tuned versions of the models widens, which is the shape a memory effect should have: at Phase 1 the store is empty, so the agent has no experience to draw on and a search returns nothing, and any later divergence has to come from what the preceding phases wrote down (or continually learn).

The average number of steps the agents take to complete a task solidifies this result even further. Both fine-tuned versions decrease their average steps as memory accumulates, $29.5 \rightarrow 26.4$ at 9B and $51.5 \rightarrow 46.0$ at 4B. Paired per task both reductions hold (-1.84 steps, $t = -2.46$ at 9B; -3.79 , $t = -3.94$ at 4B), and the 9B base does not move at all (-0.27 , $t = -0.31$). The 4B base is the one exception: it also gets faster (-2.85 , $t = -3.11$), so at that size only about a step of the improvement is attributable to memory rather than to the phase position itself. Together the two rows say that a fine-tuned agent with a populated store solves more tasks and spends fewer actions doing it. This shows that our methodology is effective in teaching models, to continually learn and increasing their downstream performance and efficiency while doing so.

The denominator check. The results in Figure 8 do not include the runs that never completed because they exceeded the time budget: they are dropped before any outcome is reported, and what they cost surfaces instead in the average step count of the previous section, where staying inside the budget is exactly what the fine-tunes’ shorter trajectories buy. If we want to be more conservative (Figure 9), we can count every one of them as a failure. The 9B fine-tune still solves tasks under that accounting, and far more of them than its baseline, 1617 against 934 at Phase 5, a factor of 1.7, because it wins twice over: it is efficient enough to finish some 2500 trajectories per phase against the base’s 1650, and it converts a higher share of the ones it does finish, so its lead widens from $+13.77$ percentage points (pp) at Phase 1 to $+15.63$ pp at Phase 5. For the 4B we do not have the same strong result: its successes are identical at both ends, 853 and 853, so the $+4.8$ pp its curve showed was the denominator shrinking rather than more work being done. It nevertheless holds a clear advantage over its own baseline, solving more tasks at every phase while that baseline declines from 711 to 650, finishing more trajectories at every phase

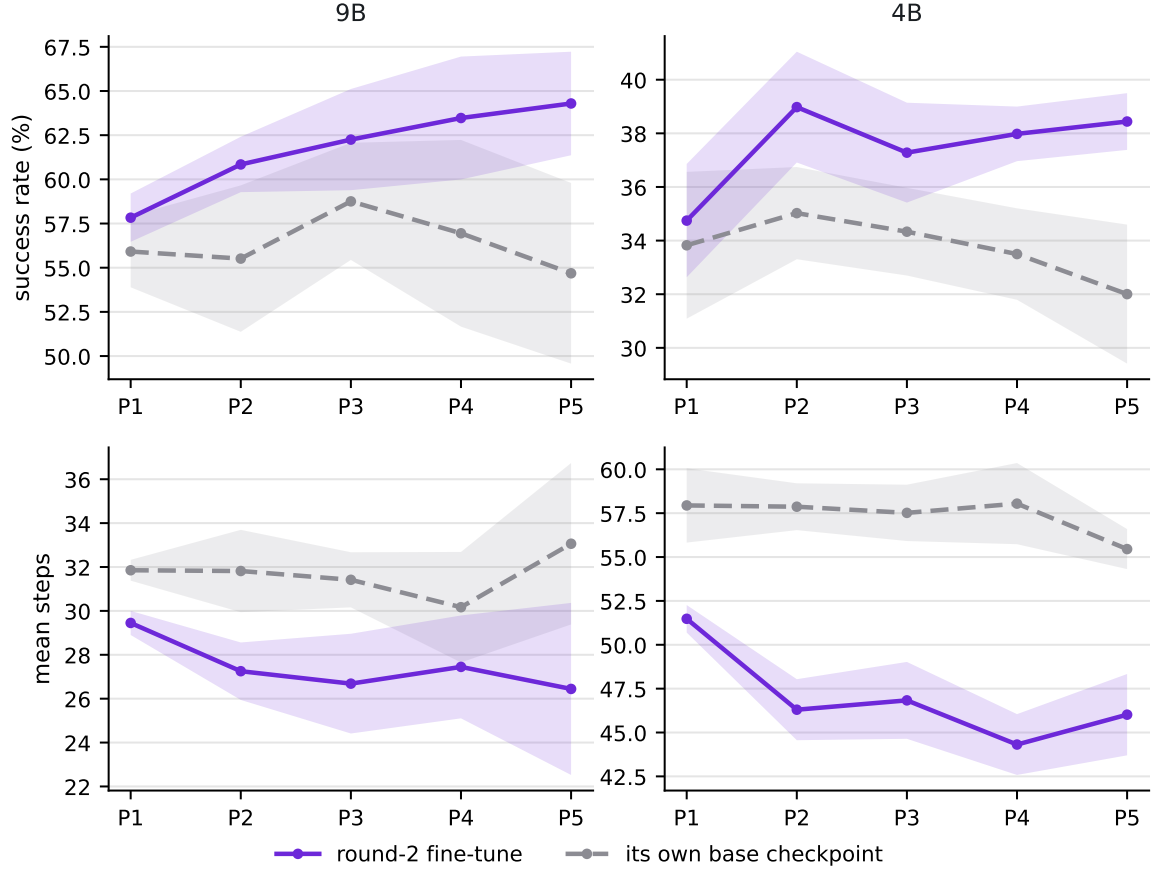


Figure 8: **The five-phase curves.** Success rate (top) and mean steps per trajectory (bottom), each model size against the checkpoint it was trained from. Lines are the mean over five replicas and bands ± 1 standard deviation across them. Only the fine-tunes move as the store fills, and they move in both rows by solving more and spending less.

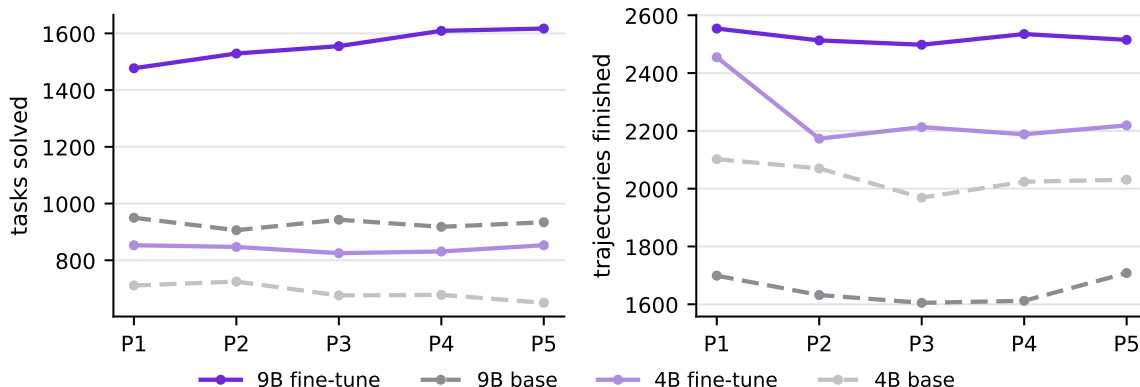


Figure 9: **Successes and attempts, in counts rather than rates.** Pooled over five replicas. The 9B fine-tune’s rising rate is backed by a rising numerator on a flat denominator; the 4B’s is not, it solves the same number of tasks at Phase 5 as at Phase 1 while finishing 236 fewer trajectories.

(2455 against 2102 at Phase 1, 2219 against 2031 at Phase 5), and reducing its average step count as the store grows, as the previous section showed.

7.2 Generalisation to unseen tasks and applications

A rising curve on tasks the model trained on could be recall rather than learning, so the evaluation set is split three ways, in increasing order of unfamiliarity. The first subset holds tasks that contributed rollouts to the training set. The second holds tasks the model has genuinely never seen, inside applications it has: no trajectory over these tasks ever entered training. The third is the strongest test: three entire applications, `paypal-my-wallet`, `xero-invoicing` and `handshake-career-exploration`, were completely withheld from the training set so the model has seen neither these tasks nor anything else in these environments.

Figure 10 replays the five phases inside each slice. The 9B fine-tune rises in all three, and by margins that do not shrink across the three subsets: +6.3 pp on trained tasks, +7.2 pp on unseen tasks and +7.0 pp on the three unseen applications. Its base checkpoint does the opposite where it matters most, falling 51.0 $\rightarrow$ 44.7% on the unseen applications. This is evidence that what the recipe teaches is a way of working and using the memory tools for continual learning rather than memorising the tasks themselves: if the phase gain were memorisation it would have to decay as the slices get harder, and instead it persists.

Table 2 gives results with a stricter measure that consider over-budget runs as failures. The 9B’s gain survives under both: +2.8, +3.4 and +5.0 pp strict against +6.3, +7.2 and +7.0 pp in the more tolerant eval, and the strict figure is largest on the unseen applications. The 4B’s gain does not survive to the strict measure: +6.0, +4.6 and +0.4 pp tolerant measures collapse to +0.5, -1.3 and -0.7 pp strict. One hypothesis is that a smaller policy has less capacity left over for behaviour that must transfer to applications it never saw, in the spirit of the

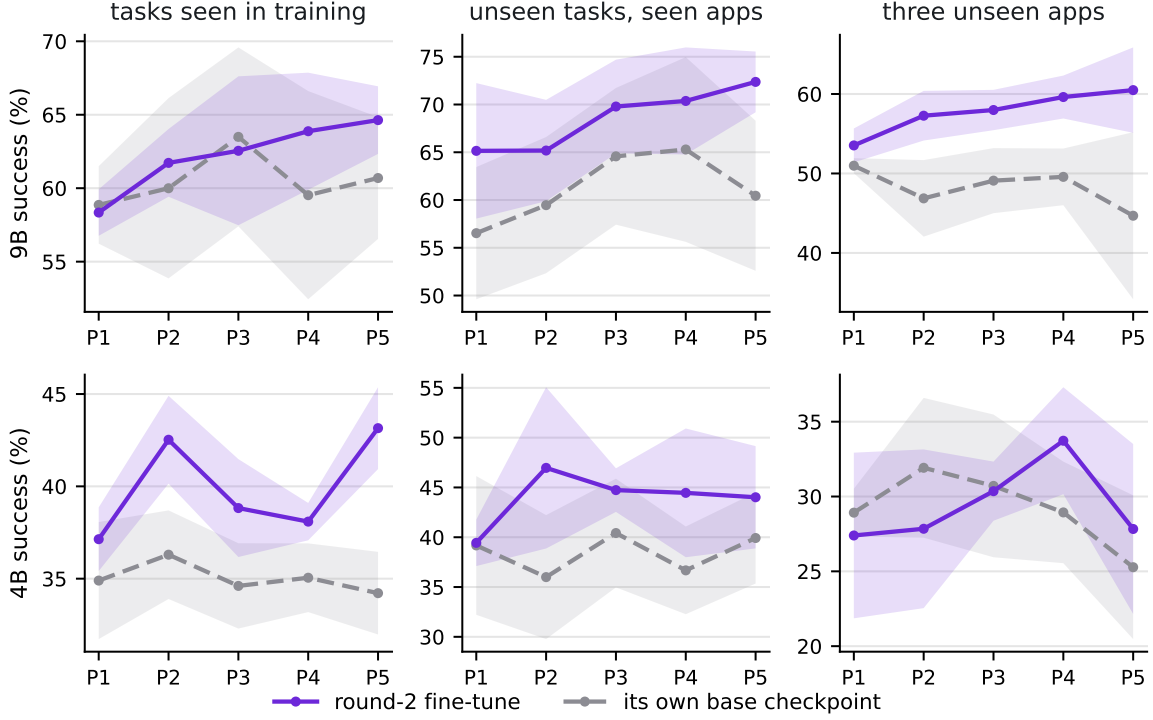


Figure 10: **The five phases inside each generalisation slice.** Columns run from tasks the model trained on, through tasks it never saw in applications it did, to three applications withheld entirely. Lines are means over five replicas, bands ± 1 standard deviation. The 9B fine-tune (top) improves in every column and by a margin that does widen from left to right, while its base falls on the unseen applications. The 4B (bottom) improves on what it trained on while its advantage is less clear on unseen applications.

scaling relationships reported by [6]. Establishing that would need a size sweep rather than a pair.

7.3 Learning to use the memory tools

In the results above we mainly focused on the macro performance of the agents after fine-tuning them for continual learning. In this section we are focusing on analysing the behavioural change. Under the policy architecture a memory operation is an ordinary tool call, just like every other action, so nothing obliges the agent to record anything; whether a store is actively built at all is a property of the policy.

In Figure 11 we report the usage of memory tools by different versions of our agents, namely the flywheel, the two fine-tuned policy and 2 not fine-tuned versions. This chart justifies the need to setup the training in the way we outlined in our recipe. Under the flywheel of Section 4 the decision to write and

Slice	tasks	Lenient		Δ	Strict		Δ
		Phase 1	Phase 5		Phase 1	Phase 5	
<i>9B round-2 fine-tune</i>							
Trained on	488	58.3	64.6	+6.3	34.6	37.4	+2.8
Unseen tasks, seen apps	121	65.1	72.4	+7.2	37.9	41.3	+3.4
Three unseen apps	197	53.5	60.5	+7.0	41.5	46.5	+5.0
<i>9B base</i>							
Three unseen apps	197	51.0	44.7	−6.3	29.3	25.3	−4.0
<i>4B round-2 fine-tune</i>							
Trained on	488	37.1	43.2	+6.0	21.4	21.9	+0.5
Unseen tasks, seen apps	121	39.4	44.0	+4.6	24.2	22.9	−1.3
Three unseen apps	197	27.4	27.8	+0.4	19.0	18.4	−0.7

Table 2: **Phase gain by slice, under both accountings.** The 9B fine-tune gains in every slice on both measures, and most on the applications it never saw; the 4B’s apparent gains do not survive counting timeouts as failures. The 9B base row is included as the control that makes the fine-tune’s unseen-application gain readable: on the same 197 tasks it moves the other way.

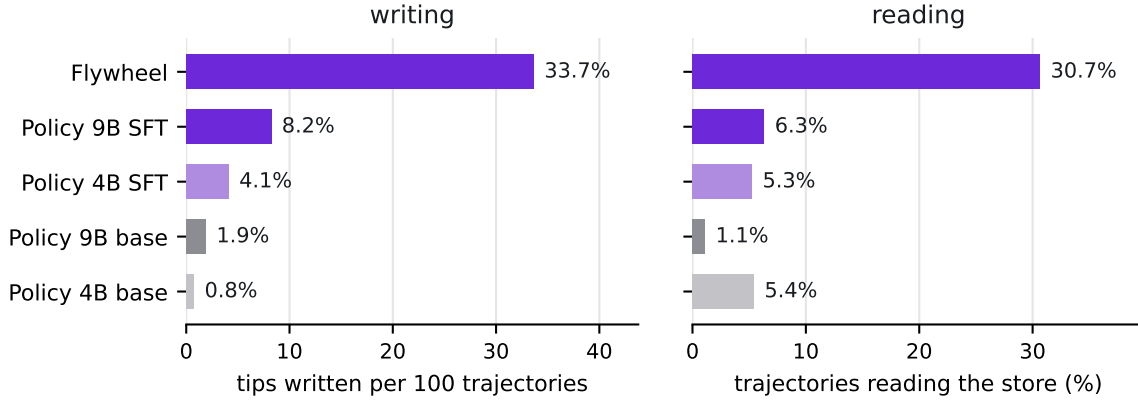


Figure 11: Left: tips written per 100 trajectories. Right: the share of trajectories in which the store was read mid-run (initial briefing excluded).

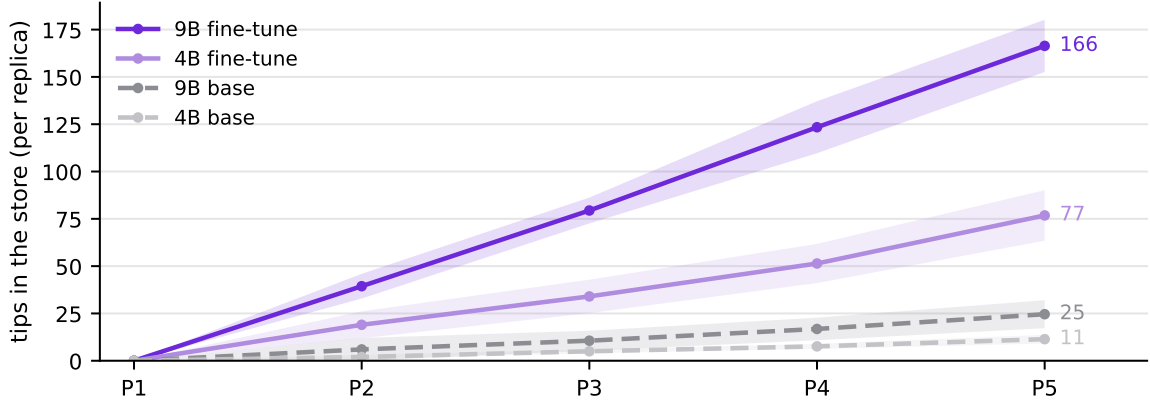


Figure 12: tip stores growth as a function of the Phases. Over time the fine-tuned agents learn how to accumulate more experience and distill that into tips to add to the store.

read from the store belongs to the manager, a model called on every step whose only job is that decision, tips are written on 33.7% of trajectories and 30.7% of them read from the store, 18 to 40 times the rate of an un-fine-tuned policy given the same tools. That gap is precisely why the flywheel exists as a data collector: it manufactures the demonstrations a policy will not produce on its own. It is not necessarily a candidate for deployment, as it is overfiring in most of the cases, reason for which we need to filter for the good rollouts, however generates some signal for when a policy should call memory operation tools. On the contrary, an un-fine-tuned checkpoint essentially never writes to the store: 1.9% of trajectories at 9B and 0.8% at 4B. After fine-tuning, that jumps to 8.2% and 4.1% respectively, thus showing that the fine-tuning effectively teaches the models to operate the writing tool. In Figure 12, over five phases and 806 tasks the base arms accumulate stores of 25 and 11 tips, against 166 and 77 for the fine-tuned versions. This is why the base versions of the agents in every figure above are flat: a 15-item store cannot distill experience over five phases completions of 806 tasks, so their curves are flat by construction and say nothing about whether memory helps. Post-training moves the write rate roughly four-fold at both sizes, and that is the mechanism behind the phase curves.

Less evidently than the writing tools as shown on the right of Figure 11, the reading operations are learned after fine-tuning. In this case the 9B versions jump from 1.1% to 6.3% of the total trajectories, while the fine-tuned 4B policy maintains an already acceptable rate. In general we would not expect an agent that masters memory tools to use the reading operation more frequently, because ideally the forced initial memory briefing (not counted as a read operation in this section) already provides the policy with enough information on the content of the tips.

At no point in the pipeline does the third operation take hold: the manager’s

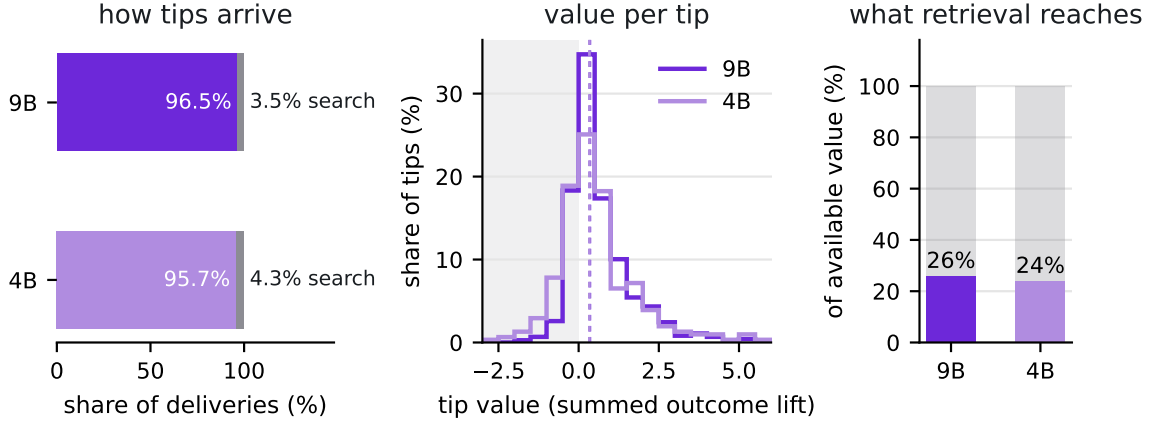


Figure 13: **What is in the store, and how much of it arrives.** Left: the share of deliveries that came from the forced briefing (coloured) against agent-initiated `search_tips` (grey). Middle: the distribution of per-tip value, normalised to each store’s own size so the two are comparable; the shaded region left of zero holds tips that made their receivers worse, and the dashed lines mark the medians, which nearly coincide. Right: the value the briefing actually delivered as a share of the best equally-sized selection available in the same store at the same moment.

deletes were rare enough that only two `delete_tip` examples survived into the 8726 training turns of round 1, and the policy trained on them fires the operation in 0.15% of trajectories in round 1 and essentially never afterwards: across all 43444 completed trajectories of the four arms evaluated here, `delete_tip` fires exactly once. Every store in this report therefore grows monotonically (as per Figure 12) and it is never curated; hence representing a limitation of our approach.

7.4 Tip quality

The sections above measured what the store did for success and how often the policy touched it. In this section we dig deeper in the tips that the models generate and try to assess their quality. Every tip is scored using Equation 1. Out of every tip processed by the policy, 96.5% at 9B and 95.7% at 4B came from the forced start-of-task injection; the tool the models were explicitly trained to call supplied the remaining 3.5% and 4.3%(Figure 13, left).

Figure 13 shows that at 9B, 77.7% of tips carry positive score against 68.1% at 4B, but the median tip is almost identical in the two stores and is > 0 ($+0.35$ and $+0.34$). Table 3 shows that both model dimensions manage to construct majority-positive tip stores, which is consistent with the tips carrying real value generated.

For the 4B version of the models 64.5% of its tips come from trajectories that did not succeed, against 34.6% at 9B, and its failure-written tips are worth $+0.51$

	9B fine-tune	4B fine-tune
Tips written	737	307
Deliveries	77 049	49 197
... via the forced briefing	96.5%	95.7%
... via <code>search_tips</code>	3.5%	4.3%
Tips with positive lift	77.7%	68.1%
Median tip value	+0.35	+0.34
25th percentile	+0.02	-0.13
Harmful tips	161 (21.8%)	98 (31.9%)
... total cost	-44.6	-53.7
Net store value	+504.5	+184.1
Share of value in the top decile	42%	44%
Written by a run that succeeded	65.4%	34.7%
... mean value of those tips	+0.70	+0.75
... mean value of failure-written tips	+0.65	+0.51

Table 3: **Tip lineage.** Value is summed outcome lift against each arm’s own Phase 1 pass rate.

against +0.75 for its success-written ones. The 9B shows the same sign with a much smaller gap, +0.65 against +0.70. The store the 4B builds is largely a record of what went wrong, written by a policy that did not find the way out, which is a mechanism consistent with the observation that an agent failure can effectively convert memory into success, thus justifying the result in Section 7.1.

What the extremes look like. The scores are more interpretable next to the text they score. Table 4 lists the three most and three least valuable tips in the 9B store. By looking at the last column is clear that the outcome of the agent that writes the tip is independent of the quality of the tip that gets generated. The best tips encode *navigation procedure* (which panel, which tab, in what order) while the worst bake *instance values* into what reads as a general rule. The best tip in the store is procedural throughout:

1) Navigate to Employers section from left sidebar, 2) Use search bar to filter by ‘Media & Entertainment’ industry, 3) Click on the employer card to access their page, 4) Click ‘Follow’... Verify each action by checking visual confirmation (button state change, toast notifications, counter increments).

The worst states a correct rule and then hard-codes that instance’s answer to it:

1) Navigate to Connected Accounts in Profile settings, identify the account connected on the same date as your Linear account (Jun 15, 2024 for Google), click Disconnect... 2) ... click Default home view dropdown, select the alphabetically last option (‘My Issues’ among: All issues, Active issues, Current cycle, ...).

Delivered to another instance of the same application, the date and the named option are simply wrong, and the receiving agent follows them. The other two harmful tips do the same thing, one naming a specific invoice number, customer and amount, the other a numeric threshold and a literal URL parameter.

	Tip title	value	deliveries	writer solved it?
<i>Three most valuable</i>				
1	Navigate to a Media & Entertainment employer, then act	+8.00	863	yes
2	RSVP to events hosted by followed employers: none found	+7.29	595	no
3	Add paragraph indent to layers with spacing > 0	+6.27	409	no
<i>Three least valuable</i>				
735	Sell Bitcoin when portfolio exceeds threshold	−1.28	41	no
736	Copy paid invoice to draft and change reference in Xero	−1.58	67	no
737	Disconnect Google account and set default home view	−1.70	337	yes

Table 4: **The extremes of the 9B store**, out of 737 tips. Value is summed outcome lift, computed following Equation 1. Two of the three best tips were written by trajectories that failed, and the worst by one that succeeded. And delivery volume does not track value, the worst tip was delivered 337 times, similarly to the third-best.

These examples are enough to show that the score assigned to the tips is indicative of their actual usefulness. Being the stores mostly populated by positively scored tips, the fine-tuned models are writing stores whose contents are, on balance, worth delivering, though the scores measure delivery outcomes rather than establishing that any individual tip caused them.

8 Ablations

In the results above we have demonstrated the effectiveness of the proposed recipe in teaching computer-use agents to learn continually. In this section we will explain the motivations behind the design of our solution through a series of ablation studies. In particular we focus on the designing choices that lead to the flywheel architecture, the policy architecture and the collection rollouts filtering technique.

8.1 Flywheel architecture design

The flywheel reached its shipped form through six different iterations, each motivated by a measured defect in the one before. Table 10 records them, since together they amount to a retrospective ablation of the design: the briefing, the manager and the retrieval front-end were each added or changed in isolation and their effects measured. The shipped design’s two least obvious choices are worth stating explicitly. The manager exists at all because V1.3 was the first variant in which a store reliably accumulated, the same threshold that Section 7.3 shows an un-fine-tuned policy failing to cross. Since the flywheel is a collector rather than a deployment candidate, it was chosen on write behaviour rather than success, it is the only variant whose injections rise with store content, 40.0 to 53.8 per phase, and whose creates decay as the store matures, 22.8 to 9.6 (with each task repetition, the agent extracts progressively fewer novel insights).

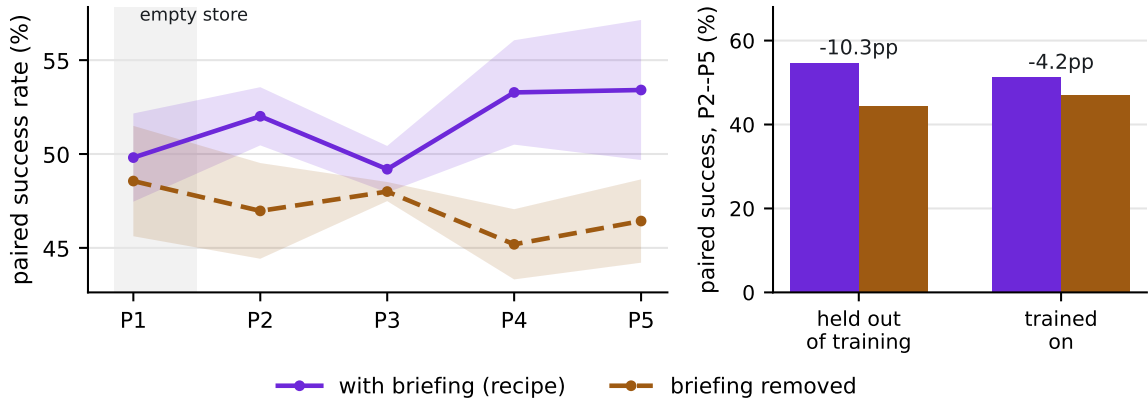


Figure 14: Left: paired success by phase, bands ± 1 standard error of the phase mean across the three replicas. Right: pooled comparison along the fine-tune’s training split.

8.2 Initial memory briefing

Letting the policy choose when to use the memory tools itself, with no help whatsoever by any callback to intervene in these decisions, would be an intuitive approach to solving the continual learning problem. Our policy architecture, however, includes an extra forced briefing over the memory store at the beginning of the trajectory. To explain this design choice we setup an ablation experiment that compares the performance of our policy design with a slightly modified version of it where the injection callback is deleted (there is no initial briefing) and the two prompt lines that assert a briefing exists are rewritten; the rest stays unchanged. The base model is the 9B fine-tuned version and the evaluation runs on 198 tasks over five phases and experiment is repeated three times, around 5 940 trajectories per arm in total. A smaller version of the full evaluation allows us to make a faster, but still confident, judgement.

Figure 14 shows the arms starting together and separating once the store fills, thus demonstrating that the forced initial briefing is necessary for the agent to improve continually; removing it costs -5.46 pp (95% confidence interval (CI) $[-8.01, -2.96]$, $p = 3 \times 10^{-5}$, 1 723 paired observations).

Table 5 (and Figure 14 on the right) demonstrate the need of the briefing at an even more granular level. Splitting the pooled comparison according to the SFT post-training split, the briefing is worth -10.3 pp on the 40 tasks held out of training against -4.2 pp on those trained on, 2.5 times as much. Intuitively one could reason that a checkpoint which has already seen a task carries the procedure in its weights and needs the briefing less, so initial injection earns its gap exactly where training did not reach.

The drop in performance of the version with no initial forced injection could

Phase	paired n	with briefing	briefing removed	gap	p
Phase 1 (<i>empty store</i>)	422	49.8%	48.6%	−1.2 pp	0.704
Phase 2	433	52.0%	46.9%	−5.1 pp	0.061
Phase 3	396	49.2%	48.0%	−1.3 pp	0.704
Phase 4	448	53.1%	45.1%	−8.0 pp	0.003
Phase 5	446	53.4%	46.4%	−7.0 pp	0.006
Phases 2–5 pooled	1 723	52.0%	46.5%	−5.5 pp	3×10^{-5}
<i>Pooled Phases 2–5, split by the fine-tune’s training split</i>					
Held out of training	360	54.7%	44.4%	−10.3 pp	0.001
Trained on	1 363	51.3%	47.1%	−4.2 pp	0.004

Table 5: **The briefing ablation, paired per task.** Success is the share of paired tasks solved; p is an exact two-sided sign test on the discordant pairs.

be an artifact of the prompt shift rather than missing information: the briefing-free arm rewrites the two prompt lines that assert a briefing exists, and the SFT post-training happened on the original system prompt. However, this element still represents a good design decision as it mirrors other emergent architectures across different popular agents regarding the use of skills. Skills are pieces of instructions regarding specific workflows that get injected at the beginning of the context of the agents to guide them in the completion of certain tasks. These skills are usually hand-crafted, in our case they are dynamically constructed by the agent tips and their retrieval (inspired by automated skill discovery [1]).

8.3 Round 1 versus round 2

Finally, we provide supporting arguments for the filtering choices made in Section 5.2. To do so, starting from the same identical checkpoint, we fine-tune the 9B model using two different filterings of the same collected rollouts. We are going to assign to round 1 the version that kept the over-long failed rollouts and to round 2 the version that excludes them, following the explanation of Section 5.2. Figure 15 shows what dropping the over-long failed rollouts bought: the level rises by nearly ten points, $48.4 \rightarrow 58.2\%$ at Phase 1, the phase gain grows from +4.30 pp ($t = 3.36$) to +6.34 pp ($t = 4.09$), and mean steps fall from 47.6 to 29.3.

The store can also be scored directly, without reference to the success curve. Each delivery of a tip is credited with the outcome-lift score of Equation 1, and summing over a tip’s deliveries gives its quality; a tip written by a run that failed, or delivered to tasks that go on to fail, scores negative. Read this way (Figure 16), the two rounds produce visibly different stores. Round 1 built 4 498 tips of which 54.9% carry positive lift and 66% were written by trajectories that themselves failed; round 2 built 737, of which 77.1% are positive and 35% come from failed runs. The two rounds differ in store size and in the filter that produced it at the same time, so this comparison cannot separate the two: what it shows is that six times fewer tips, selected differently, did not cost performance and coincided with a larger phase gain.

Table 6 records all the failure modes that the Round 2 filter removed. Round 1’s fine-tune exhibited a family of tool-use pathologies absent from the

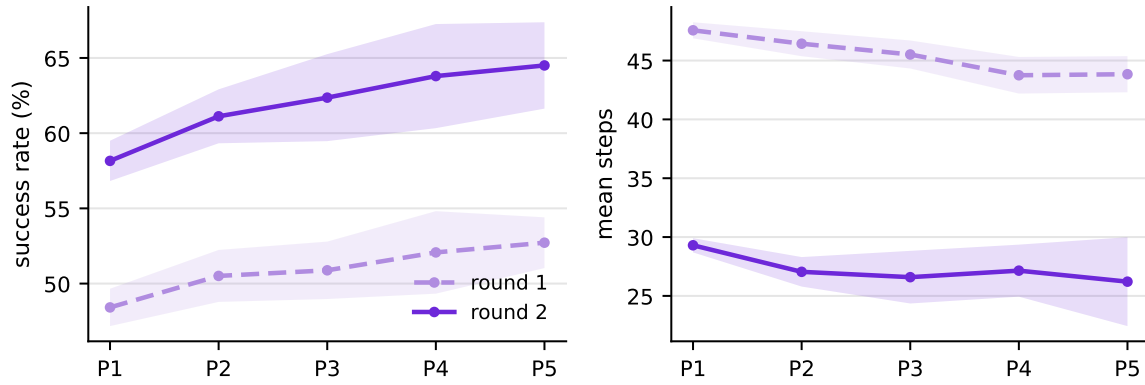


Figure 15: **The two dataset rounds on the same protocol.** The filter improves the performance, the growth and the steps count at once.

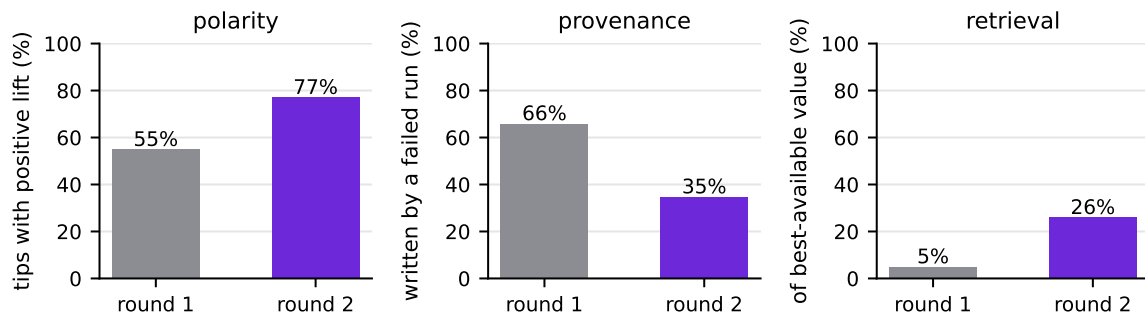


Figure 16: **What the length filter did to the store.** Left: the share of tips whose deliveries carry positive outcome lift. Middle: the share written by a trajectory that itself failed. Right: the value the briefing actually retrieves as a share of the value of the best eight tips available in the same index at the same moment.

Trajectories exhibiting	Round 1	Round 2
Save storm (≥ 5 saves)	1.22%	0.00%
Search flood (≥ 10 searches)	7.41%	0.00%
Verbatim query loop	11.54%	0.01%
Collapsed retrieval depth ($k \leq 1$)	28.15%	0.11%
Step-budget exhaustion	26.79%	6.72%
Memory never used	52.42%	87.38%

Table 6: **Tool-use pathologies, before and after the length filter.** Every mode in the upper block is absent from the un-fine-tuned checkpoint and was therefore introduced by post-training; all of them disappear once the over-long failed rollouts are dropped. The last row is the cost of that fix.

un-fine-tuned model, and all of them trace to the same source: 38 of 321 short-listed rollouts had failed, averaging 43 steps against 22, and so supplied 20.7% of the training examples. Dropping them removed the failure modes almost entirely (the verbatim query loop and the collapsed retrieval depth survive at 0.01% and 0.11%), and the last row of the same table records a property that emerged with them: the share of trajectories that never touch memory at all rises from 52.4% to 87.4%. Reading that as a cost would assume a policy has to consult its memory constantly to benefit from it, which these results do not support. The briefing already delivers what most tasks need, and the round-2 policy reaches for the tools only on the minority of runs where the briefing was not enough, which is the behaviour a well-calibrated use of memory should show.

9 Limitations

The demonstration generalises well to tasks and applications the models never saw during training, but it is a browser result throughout. Only 6% of the collected training data is OSWorld rollouts, so the fine-tune is effectively specialised to WebArena Infinity rather than to computer use in general. Balancing the two suites and blending them into an existing general-purpose mixture would help reducing the overfitting to this dataset that fails to generalise to other benchmarks.

Two `delete_tip` examples survived into 8 726 training turns, and the resulting policy fires the operation once in 43 444 trajectories. Every store in this report therefore only grows, which caps how long a store can stay useful and leaves an unused action occupying space in the prompt. Fixing this would require directing the manager of the flywheel to produce more deletion examples.

10 Conclusion and future work

In this work we presented a methodology to teach computer-use agents how to use memory tools to enable their continual learning abilities. Our contributions include an empirical way to evaluate continual learning for agents, a scoring technique to assess the quality of the tips created by the models, an

automatic collection and fine-tuning pipeline. We showed the effectiveness of this setup on multiple fronts. A 9B policy trained with the recipe improves monotonically across five passes over the same task set, from 57.8% to 64.3%, while the checkpoint it was fine-tuned from drifts downwards under the identical protocol; it solves 140 more tasks in absolute terms and spends fewer steps doing it. That gain is even larger on tasks the model never saw, demonstrating the generalisation ability of our approach. Expanding our approach to smaller model dimensions we demonstrate the transferability of our approach despite the lower performance compared to the 9B arm. We demonstrate the effectiveness of our approach also analysing qualitatively the tips that the final models eventually generate, assess that the fine-tuned agents manage to generate high-quality tip stores. Finally, we justified the design choices behind our approach by presenting some ablation results.

Future work. Three directions follow directly from the results above. The first is the cheapest: the retrieval budget was fixed at eight tips for the briefing and five tips for the search tools throughout all the experiments and it was never tuned. A sweep over that budget is a hyperparameter search that would further enhance the performance. The second is to fold the dataset into a general-purpose mixture and rebalance the two benchmarks within it, which addresses the difficulty to generalise to other benchmarks as we discussed in the limitation. The third is the natural extension of the whole pipeline. Supervised fine-tuning can only imitate the manager’s decisions that survived the filtering, however the tip score of Section 7.4 is already a per-tip credit assignment computed from outcomes, it is precisely the signal a reinforcement-learning (RL) objective could benefit from. Optimising against it directly using an RL objective, rather than against a fixed set of demonstrations, would let the policy discover when consulting memory is worth a turn instead of being told.

References

- [1] Salaheddin Alzubi et al. *EvoSkill: Automated Skill Discovery for Multi-Agent Systems*. 2026. arXiv: 2603.02766 [cs.AI]. URL: <https://arxiv.org/abs/2603.02766>.
- [2] Anthropic. *How Claude remembers your project*. Tech. rep. Anthropic, 2026. URL: <https://code.claude.com/docs/en/memory#auto-memory>.
- [3] H Company. *Holo3.1: Fast & Local Computer Use Agents*. Tech. rep. H Company, 2026. URL: <https://hcompany.ai/holo3.1>.
- [4] Gaodan Fang et al. *Trajectory-Informed Memory Generation for Self-Improving Agent Systems*. 2026. arXiv: 2603.10600. URL: <https://arxiv.org/abs/2603.10600>.
- [5] Prime Intellect. *Prime Agent: A self-improving RLM agent*. Tech. rep. Prime Intellect, 2026. URL: <https://www.primeintellect.ai/blog/prime-agent>.
- [6] Jared Kaplan et al. “Scaling laws for neural language models”. In: *arXiv preprint arXiv:2001.08361* (2020).

- [7] Seth Karten et al. *Continual Harness: Online Adaptation for Self-Improving Foundation Agents*. 2026. arXiv: 2605.09998. URL: <https://arxiv.org/abs/2605.09998>.
- [8] Omar Khattab and Matei Zaharia. “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT”. In: *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2020, pp. 39–48.
- [9] James Kirkpatrick et al. “Overcoming catastrophic forgetting in neural networks”. In: *Proceedings of the National Academy of Sciences* 114.13 (2017), pp. 3521–3526. DOI: 10.1073/pnas.1611835114. eprint: <https://www.pnas.org/doi/pdf/10.1073/pnas.1611835114>. URL: <https://www.pnas.org/doi/abs/10.1073/pnas.1611835114>.
- [10] Mem0. *AI Memory Management for LLMs and Agents*. Tech. rep. Mem0, 2026. URL: <https://mem0.ai/blog/ai-memory-management-for-llms-and-agents>.
- [11] OpenClaw. *Memory architecture*. Tech. rep. OpenClaw, 2026. URL: <https://docs.openclaw.ai/concepts/memory-architecture>.
- [12] Qwen Team. *Qwen3.5: Towards Native Multimodal Agents*. Feb. 2026. URL: <https://qwen.ai/blog?id=qwen3.5>.
- [13] Nous Research. *Persistent Memory*. Tech. rep. Nous Research, 2026. URL: <https://hermes-agent.nousresearch.com/docs/user-guide/features/memory>.
- [14] Raphaël Sourty. *NextPlaid: Multi-vector search, from database to coding agents*. 2026. URL: <https://github.com/lightonai/next-plaid>.
- [15] Harsh Trivedi et al. “AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2024, pp. 16022–16076.
- [16] Tianxin Wei et al. *Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory*. 2025. arXiv: 2511.20857. URL: <https://arxiv.org/abs/2511.20857>.
- [17] Wenyi Wu et al. *Auto-scaling Continuous Memory for GUI Agent*. 2025. arXiv: 2510.09038. URL: <https://arxiv.org/abs/2510.09038>.
- [18] Tianbao Xie et al. “OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments”. In: *Advances in Neural Information Processing Systems*. 2024.
- [19] Sikuan Yan et al. *Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning*. 2025. arXiv: 2508.19828. URL: <https://arxiv.org/abs/2508.19828>.
- [20] Junhao Zheng et al. “Lifelong learning of large language model based agents: A roadmap”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2026).
- [21] Shuyan Zhou. “WebArena-Infinity: Generating Browser Environments with Verifiable Tasks at Scale”. In: (Mar. 2026). URL: <https://webarena.dev/webarena-infinity/>.

A Full rollouts of the motivating example

All five independent rollouts of the same policy on `libreoffice_calc_40`, one per figure and at a larger scale than the three shown in the main text. The five come from the same weights and the same prompt; the procedure that solves the task in 15 steps was available to every one of them.

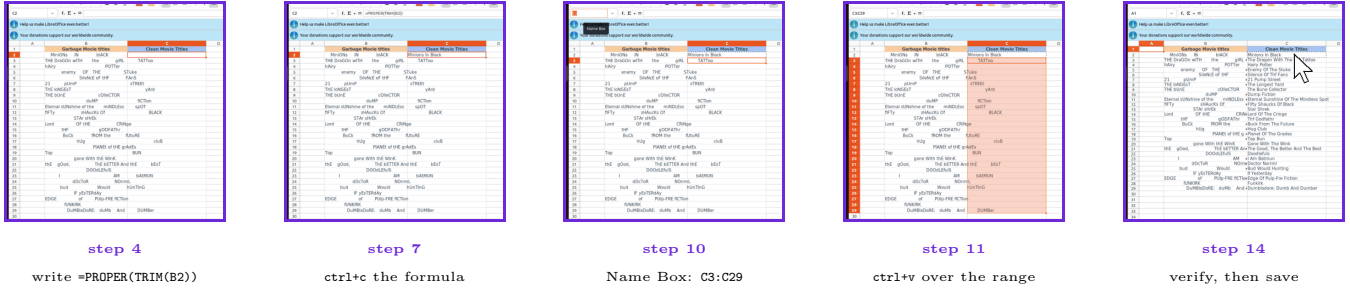


Figure 17: **Rollout 4: solved in 15 steps.** The only successful attempt: one `PROPER(TRIM(...))` formula, one copy, one range selection through the Name Box and one paste.

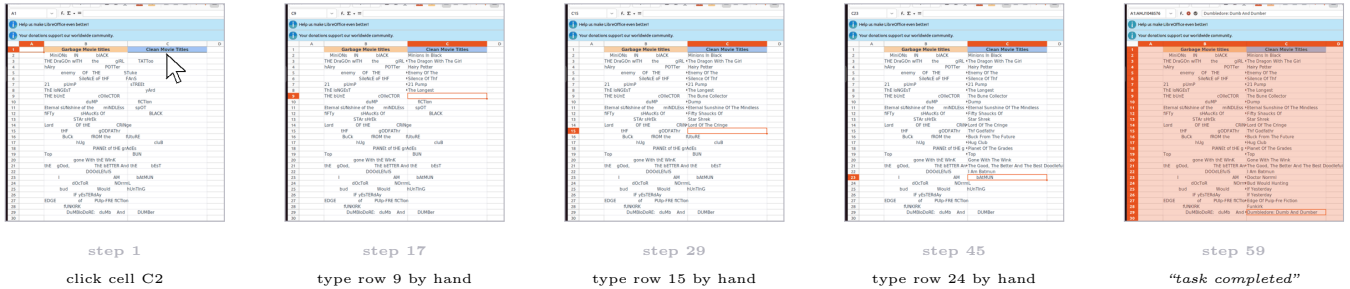
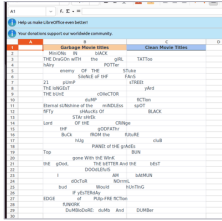
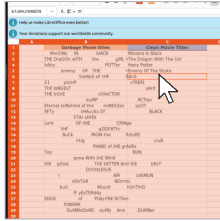


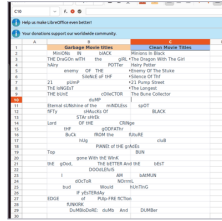
Figure 18: **Rollout 1: 59 steps, corrupted.** 28 cells typed by hand. Nine rows lose their trailing word and one title is written twice, shifting the tail of the column. The agent reports success.



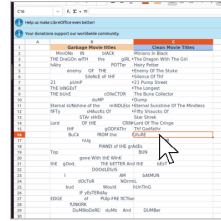
step 1
plan the row loop



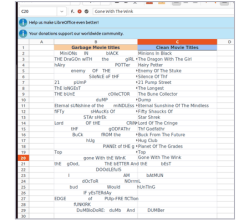
step 22
click C6



step 50
type row 10 by hand

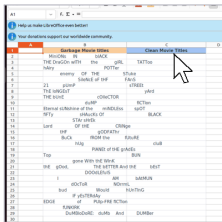


step 78
double-click C16

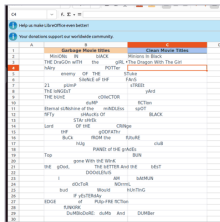


step 99
"20/28 rows"

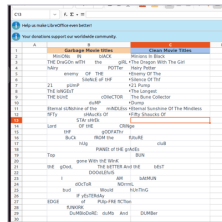
Figure 19: **Rollout 2: 99 steps, incomplete.** The same cell-by-cell strategy, abandoned at row 20 of 28 when the step budget ran out.



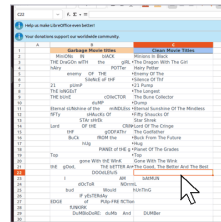
step 1
click cell C2



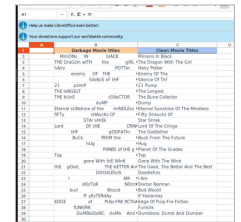
step 4
type row 4 by hand



step 16
type row 13 by hand

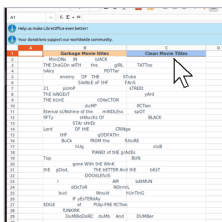


step 31
click C23

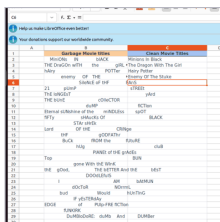


step 45
"all 28 titles"

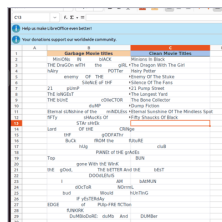
Figure 20: **Rollout 3: 45 steps, corrupted.** Reaches the end of the column but mistypes several titles along the way.



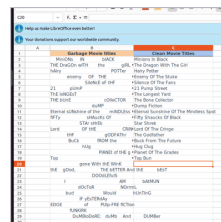
step 1
click cell C2



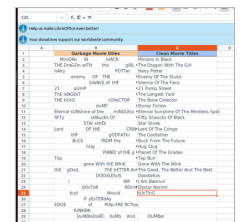
step 22
type row 6 by hand



step 50
type row 13 by hand



step 78
type row 20 by hand



step 100
"partially complete"

Figure 21: **Rollout 5: 100 steps, incomplete.** Exhausts the step budget partway down the column.

B Training details and loss curves

The composition of the supervised dataset, the hyperparameters every run shares, the four runs themselves and their loss curves, referenced from Section 5.

	Round 1	Round 2
Over-long failed rollouts	kept	dropped
Training examples (policy turns)	8 726	7 105
Total tokens	160.9 M	126.6 M

Table 7: **The two SFT dataset rounds.** Token counts are measured over the rollouts contained in the dataset. Each rollout has a maximum length of 32 768 tokens. The two rounds differ only in the length filter of Section 5.2; round 2 is the dataset behind the final model.

Setting	Value
Base model	Ho1o-3.1-9B (Ho1o-3.1-4B for the size arm)
Learning rate	1×10^{-6} (2×10^{-6} for the 4B)
Epochs	1
Maximum sequence length	32 768
Long-sequence handling	truncate from the beginning
Micro-batch size per GPU	1
Gradient accumulation steps	1
Ulysses sequence-parallel size	1
Gradient checkpointing	enabled
Trainable assistant messages	1 per example
Mixture weight	1.0 (standalone)

Table 8: **SFT configuration.** A single epoch over the dataset, at a learning rate an order of magnitude below the base model’s pretraining schedule, is enough to move tool-firing behaviour; each run is deliberately conservative in order to avoid catastrophic forgetting.

	Round 1, 9B	Round 2, 9B	Round 2, 4B
Base model	Holo-3.1-9B	Holo-3.1-9B	Holo-3.1-4B
Dataset	round 1	round 2	round 2
Learning rate	1×10^{-6}	1×10^{-6}	2×10^{-6}
Nodes $\times$ GPUs	1×8	2×8	2×8
Optimisation steps	670	260	260
Tokens per step	0.18 M	0.47 M	0.46 M
Wall-clock	79 min	37 min	27 min
Final loss (smoothed)	0.26	0.21	0.18

Table 9: **The three training runs.** Steps and wall-clock are as recorded; the final loss is the mean of the last five logged points. The round-2 9B run is the final model.

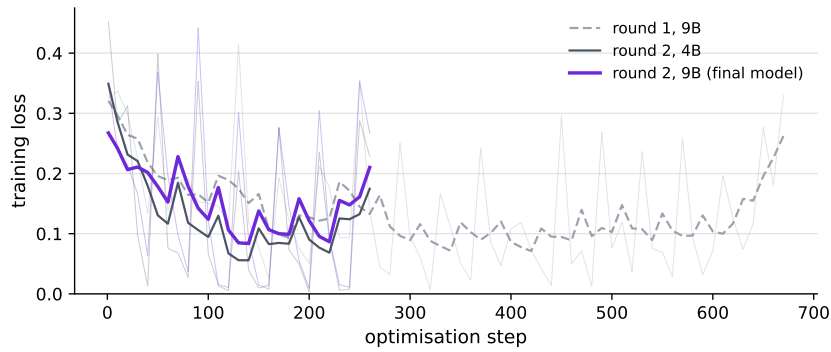


Figure 22: **Training loss.** Faint lines are the raw per-step loss, solid lines a rolling mean over five logged points (logging is every ten steps). Round 1 is dashed because it is the only run on a different dataset.

C Flywheel iterations

The six harness iterations discussed in Section 8, with the measurement each design decision was taken on.

Variant	What changed	Measured effect	What it showed
V1	per-step retrieval	own-set tips +10.0 pp over a 60.0% baseline, falling to −2.9 pp once the store also held a second task set ($n = 70$)	Retrieval helps only when what arrives is relevant; diluting the store reversed the gain.
V1.2	inject once at start	<i>not measured</i>	The substitution was adopted on cost grounds. V1.3.2 later priced per-step retrieval at $1.7\times$ the phase-1 wall-clock for no gain.
V1.3	briefing + manager	~1,960 tips written and ~1,440 injected over five phases, from 94% passthrough decisions	The first variant in which a store reliably accumulated; the curve rises steeply in Phase 2 and then plateaus.
V1.3.1	write hygiene	duplicate titles 11–21% → 1–4%, deletes 22 against 1, 85% of tips injected at least once, but Phase 5 57.2% against 62.8%	Every hygiene target was hit and none of it bought success: hygiene was not the binding constraint.
V1.3.2	LLM query + relevance filter	lift per delivery +0.028–0.051, 3–10 times V1.3.1; filter keeps 3–5% of $k = 15$ and 60–70% of briefings are empty	Targeting the briefing raised the value of what was delivered, at the price of usually delivering nothing.
V1.3.3	true once-at-start briefing; 122B manager	Phase 1 → Phase 5 +0.33 pp ($t = 0.17$); retrieved quality 0.058 against 0.59 available; manager injections rise 40.0 → 53.8 as the store fills	Moderate on success, but the only variant that produces useful signal.

Table 10: **The flywheel iterations, read as an ablation.** Each row changes one element of the harness, and the third column gives the measurement the decision was taken on. The effects are measured on 84–86 desktop tasks rather than the full evaluation.

D Conversion example

One collected rollout step as the flywheel recorded it and as the converter re-expresses it for training, referenced from Section 5.

COLLECTED — the flywheel harness, one step, two models

```
% the acting model. Its action list has no memory tools.
{"note": "The Enable Auto Drafts toggle is still ON ...",
 "thought": "The Auto Drafts toggle is still enabled. I need to click it again ...",
 "tool_call": {"tool_name": "click_web", "x": 945, "y": 675}}

% the supervising model, same step. A verdict, not an action.
{"step": 13,
 "action": "create",
 "category": "general",
 "reasoning": "The agent successfully located the Auto Drafts setting in the Reminders
tab ...",
 "note": "Auto Drafts setting found in Reminders tab ...",
 "created_tip_id": "5da04f69-8346-4f3c-aa77-f98e8d0f666d"}
```

REQUIRED — what a V3 policy emits: one model, one object

```
{"note": "Auto Drafts setting found in Reminders tab ...",
 "thought": "I am about to disable the Auto Drafts setting. I know that this setting is
located in the Reminders tab, not in General as one might expect ...",
 "tool_call": {"tool_name": "save_tip",
               "title": "Auto Drafts setting location in Superhuman",
               "content": "Auto Drafts setting is located in the Reminders tab of
Settings, not in General ...",
               "task_pattern": "Disabling Auto Drafts in Superhuman Mail settings"}}
```

WHAT THE CONVERTER DOES

```
action → tool_name a verdict becomes a call: create ↦ save_tip, inject ↦
search_tips, delete ↦ delete_tip
reasoning → thought re-voiced from third to first person, keeping every detail, turning
a verdict into an intention
note → note copied verbatim from the supervisor
created_tip_id → title, content, task_pattern the record holds only an id; the tip's
text is looked up and inlined as the call's arguments
the category the supervisor emitted, which is not one of the four the tool accepts

the acting model's turn is kept unchanged, as a separate training example; the memory turn
is inserted beside it
```

Figure 23: **Conversion steps.** The collected steps above come from a WebArena Infinity rollout in the collection campaign, and the required form is that trajectory's actual training example. The flywheel splits a step across two models (the manager and the actor) thus conversion has to fuse the two into the single object a policy emits, which is a change of speaker and of form rather than a change of format. Colours pair each field with what it becomes.

E Tip texts

The complete text of the six tips of Table 4: the three most and three least valuable of the 737 in the 9B store, quoted verbatim as the policy wrote them. Value is summed outcome lift and *written by a solved run* records whether the trajectory that created the tip went on to succeed.

Rank 1 of 737 (value +8.00, 863 deliveries, category **strategy**, written by a solved run: yes) *Navigate to Media & Entertainment employer and complete all required actions*

To complete this task: 1) Navigate to Employers section from left sidebar, 2) Use search bar to filter by 'Media & Entertainment' industry, 3) Click on the employer card (Spotify) to access their page, 4) Click 'Follow' button to follow the employer, 5) Click 'Jobs' tab to view job listings, 6) Click bookmark icon on active job card to save it, 7) Navigate to Events section, 8) Find and click RSVP button on the virtual career fair event. Verify each action by checking visual confirmation (button state change, toast notifications, counter increments).

Rank 2 of 737 (value +7.29, 595 deliveries, category **general**, written by a solved run: no) *RSVP to upcoming events hosted by followed employers - no matching events found*

When tasked with RSVPing to upcoming events hosted by employers you follow: 1) First identify all employers you follow (Google, Microsoft, Apple, Meta, Stripe, Tesla, Bain & Company, Anthropic in this case), 2) Then check all upcoming events, 3) If no upcoming events are hosted by followed employers, no RSVP actions are needed. In this case, the only upcoming events were hosted by Stanford Career Center (not an followed employer) and Salesforce (not followed). The Apple Design Speaker Series was hosted by Apple (a followed employer) but was a past event, not upcoming.

Rank 3 of 737 (value +6.27, 409 deliveries, category **strategy**, written by a solved run: no) *Add paragraph indent to layers with spacing > 0*

To add paragraph indent of 24px to text layers with paragraph spacing > 0: 1) Select each text layer from the Layers panel, 2) Open Typography panel (click Typography in left sidebar), 3) Click Details tab to see Paragraph spacing and Paragraph indent fields, 4) If Paragraph spacing > 0, click on Paragraph indent input field and type 24, 5) Press Enter to confirm. In this file, only Body Text layer had paragraph spacing = 16px (> 0), so only that layer needed the 24px indent applied. All other layers (Page Title, Section Header, Feature List, Pricing Tiers, Call to Action, Copyright Notice) had paragraph spacing = 0px and required no change.

Rank 735 of 737 (value -1.28, 41 deliveries, category **strategy**, written by a solved run: no) *Sell Bitcoin when portfolio exceeds threshold*

When crypto portfolio exceeds 7,000 threshold: 1) Navigate to Cryptocurrency section from left sidebar, 2) Click Sell button on Bitcoin card, 3)

Enter amount in USD in sell modal, 4) Confirm sale via URL parameter (sell-crypto-amount=X), 5) If portfolio $\leq 7,000$, buy instead of sell. Then set debit card spending limit: 1) Click PayPal Debit Card in sidebar, 2) Click Change next to Daily Spending Limit, 3) Enter new limit, 4) Click Update, 5) Verify confirmation message.

Rank 736 of 737 (value -1.58, 67 deliveries, category **strategy**, written by a solved run: no) *Copy paid invoice to draft and change reference in Xero*

To copy a paid invoice to a new draft and change its reference: 1) Navigate to Invoices > Paid tab, 2) Click on the highest value paid invoice (INV-0064 for Northern Territory Power Corp at \$23,100.00), 3) Click the 'Copy' button at the top right, 4) Click 'Edit' button to enable edit mode, 5) Modify the Reference field to the new value ('REBILL-2026'). The new draft invoice will be created with the updated reference.

Rank 737 of 737 (value -1.70, 337 deliveries, category **strategy**, written by a solved run: yes) *Disconnect Google account and set default home view*

To complete this task: 1) Navigate to Connected Accounts in Profile settings, identify the account connected on the same date as your Linear account (Jun 15, 2024 for Google), click Disconnect, confirm in the modal. 2) Go to Preferences > General section, click Default home view dropdown, select the alphabetically last option ('My Issues' among: All issues, Active issues, Current cycle, Favorited Projects, Favorited Views, Inbox, My Issues).